

Déploiement d'une Application Web Full-Stack

Docker Compose & Kubernetes (K3s)

Application : Simple 3-Tier Tasks Web App

Stack : React / Express.js / MongoDB / Nginx

Rapport technique de projet DevOps - K3s

Outils : Docker/Compose CLI, kubectl, K3s, VS Code, Git

Date : Mai 2026

Table des matières

Résumé / Executive Summary	3
1 Introduction	4
1.1 Présentation de l'application	4
1.2 Architecture globale	4
1.3 Choix technologiques	4
2 Préparation de l'environnement	5
2.1 Outils installés	5
2.2 Configuration de la machine hôte	5
2.3 Installation du cluster K3s	5
2.3.1 Topologie réseau	5
2.3.2 Configuration control-plane	5
2.3.3 Workers	6
2.3.4 Difficultés rencontrées	6
2.4 Versioning (Git)	6
3 Phase 1 : Déploiement avec Docker Compose	7
3.1 Structure du projet	7
3.2 Best practices appliquées	7
3.2.1 Versions explicites des images	7
3.2.2 Réseaux personnalisés	7
3.2.3 Volumes nommés	7
3.2.4 Variables d'environnement et fichier .env	7
3.2.5 Healthchecks et dépendances	8
3.2.6 Limites de ressources	8
3.2.7 Commandes utilisées	8
4 Phase 2 : Déploiement sur Cluster Kubernetes (K3s)	9
4.1 Architecture du cluster K3s	9
4.2 Commandes d'application des manifests	10
4.3 Vérification des déploiements	10
4.3.1 3 Deployments distincts (ou StatefulSet pour la DB)	10
4.3.2 Resource requests et limits définis sur tous les conteneurs	10
4.3.3 Liveness Probe et Readiness Probe configurées	10
4.3.4 Stratégie de déploiement RollingUpdate bien paramétrée	11
4.3.5 Conteneurs exécutés en non-root (securityContext)	11
4.4 Vérification des Services et Exposition	12
4.4.1 Services ClusterIP pour backend et base de données	12

4.4.2	Service pour exposer le frontend (NodePort / LoadBalancer ou Ingress)	12
4.4.3	Secrets pour tous les éléments sensibles	13
5	Tests et Validation	14
6	Difficultés rencontrées et solutions apportées	15
7	Conclusion et perspectives	16
	Annexes	17

Résumé / Executive Summary

Ce rapport présente le déploiement d'une application web trois-tiers (gestionnaire de tâches) avec Docker Compose pour le développement local, puis Kubernetes (K3s) pour la production sur un cluster de trois nœuds. L'application a été choisie pour sa simplicité, permettant de se concentrer sur les bonnes pratiques DevOps.

Technologies : React (Vite) + Express.js + MongoDB + Nginx, orchestrées par Docker Compose et K3s.

Résultats : Déploiement fonctionnel avec multi-stage builds, utilisateurs non-root, limites de ressources, healthchecks, probes, secrets externalisés, NetworkPolicies, Ingress et HPA.

Chapitre 1

Introduction

1.1 Présentation de l'application

L'application choisie est un **gestionnaire de tâches simple** (3-Tier Tasks Web App). Sa simplicité délibérée permet de se concentrer sur la complexité du déploiement.

Fonctionnalités : CRUD de tâches via interface React, API REST Express.js, persistance MongoDB.

1.2 Architecture globale

Architecture trois-tiers :

1. **Frontend** : React servi par Nginx (port 80/8080)
2. **Backend** : API REST Express.js (port 3000)
3. **Database** : MongoDB (port 27017)

1.3 Choix technologiques

- **React + Vite** : build rapide, assets optimisés
- **Express.js** : backend léger et performant
- **MongoDB** : NoSQL documentaire adapté aux données flexibles
- **Nginx** : serveur statique et reverse proxy
- **Docker Compose** : orchestration locale simple
- **K3s** : Kubernetes allégé pour environnements limités

Chapitre 2

Préparation de l'environnement

2.1 Outils installés

- Docker Engine et Docker Compose CLI
- kubectl (client Kubernetes)
- K3s (distribution allégée)
- VS Code (édition des manifests)
- Git (versioning)

2.2 Configuration de la machine hôte

Machine hôte Windows 16 Go RAM, 3 VMs Debian sous VMware Workstation :

VM	Rôle	IP host-only	IP bridged
k3s-cp	Control-plane	192.168.204.129	192.168.1.20
k3s-w1	Worker 1	192.168.204.130	—
k3s-w2	Worker 2	192.168.204.131	—

TABLE 2.1 – Configuration des VMs

2.3 Installation du cluster K3s

2.3.1 Topologie réseau

- **Host-only** (192.168.204.0/24) : communication interne cluster (Flannel, K3s)
- **Bridged** (192.168.1.0/24) : accès LAN externe pour le control-plane

2.3.2 Configuration control-plane

/etc/rancher/k3s/config.yaml :

```
1 node-ip: 192.168.204.129
2 node-external-ip: 192.168.1.20
3 flannel-iface: <interface-host-only>
4 tls-san:
5   - 192.168.204.129
```

```
6 - 192.168.1.20
```

2.3.3 Workers

Les workers rejoignent via le control-plane host-only :

```
1 node-ip: 192.168.204.130
2 flannel-iface: <interface-host-only>
```

2.3.4 Difficultés rencontrées

- **Clonage VMs** : modification hostname, IP, /etc/machine-id, clés SSH
- **Mauvaise interface** : configuration explicite de `node-ip` et `flannel-iface`
- **Changement de sous-réseau** : mise à jour IP control-plane, régénération token, réinstallation agents

2.4 Versioning (Git)

Branche `main` pour le code stable, fichiers sensibles exclus via `.gitignore` et `.dockerignore`.

Chapitre 3

Phase 1 : Déploiement avec Docker Compose

3.1 Structure du projet

```
1 .
2 |-- react-tasks-frontend/      # React (Vite) + nginx.conf
3 |-- express-tasks-backend/    # Express + Dockerfile + init/
4 |-- docker-compose.yml
5 |-- .env
6 |-- .dockerignore
7 |-- screenshots/
```

3.2 Best practices appliquées

3.2.1 Versions explicites des images

Tag `mongo:7.0` utilisé au lieu de `latest` pour garantir la reproductibilité.

3.2.2 Réseaux personnalisés

- **frontend-net** : frontend ↔ backend
- **backend-net** : backend ↔ database

L'isolation empêche le frontend d'atteindre directement la base de données.

3.2.3 Volumes nommés

- **mongo-data** : volume nommé pour la persistance des données
- **Bind mount** : script d'initialisation monté en lecture seule (`:ro`)

3.2.4 Variables d'environnement et fichier `.env`

Toutes les variables sensibles sont externalisées dans `.env` :

```
1 DB_ROOT_USER=...
2 DB_ROOT_PASSWORD=...
3 DB_NAME=...
```



```

4 DB_APP_USER=...
5 DB_APP_PASSWORD=...
6 PORT=3000
7 VITE_API_URL=/api

```

Le fichier `.env` est exclu de Git via `.gitignore`.

3.2.5 Healthchecks et dépendances

- Database : mongosh avec `db.runCommand({ping: 1})`
- Backend : HTTP `/health` (via Dockerfile)
- Frontend : HTTP `/` (via Dockerfile)
- `depends_on` avec `condition: service_healthy` pour l'ordre de démarrage

3.2.6 Limites de ressources

Service	CPU	Mémoire	Restart
Database	0.5	512m	unless-stopped
Backend	0.5	256m	unless-stopped
Frontend	0.25	128m	unless-stopped

TABLE 3.1 – Limites de ressources Docker Compose

3.2.7 Commandes utilisées

```

1 # Build et démarrage
2 docker compose up -d --build
3
4 # Verification des services
5 docker compose ps
6 docker compose logs -f backend
7
8 # Statistiques runtime
9 docker stats

```

Chapitre 4

Phase 2 : Déploiement sur Cluster Kubernetes (K3s)

4.1 Architecture du cluster K3s

Le cluster est composé de 3 nœuds Debian sous VMware Workstation :

Nœud	Rôle	IP interne	État
k3s-cp	Control-plane	192.168.204.129	Ready
k3s-w1	Worker 1	192.168.204.130	Ready
k3s-w2	Worker 2	192.168.204.131	Ready

TABLE 4.1 – État du cluster K3s

Best Practice	Application
Namespace dédié	tasks-app
Labels cohérents	app.kubernetes.io/name, app.kubernetes.io/component
Resource requests/limits	CPU et mémoire sur tous les conteneurs
Liveness/Readiness probes	HTTP pour frontend/backend, exec pour MongoDB
RollingUpdate	maxSurge=1, maxUnavailable=0
Utilisateur non-root	runAsUser : 1001 (app), 999 (MongoDB)
Replicas critiques	2 pour frontend et backend
Secrets externalisés	db-secrets injectés via env vars
ConfigMaps	app-config, frontend-config, mongo-init-script
PVC + StorageClass	mongo-data (5Gi, app-local-path)
Ingress	Traefik sur tasks.local
HPA	backend scale 2–5 replicas selon CPU 70%
NetworkPolicies	Isolation frontend/backend/database

TABLE 4.2 – Best practices appliquées sur K3s

4.2 Commandes d'application des manifests

```

1 # Application séquentielle des manifests
2 kubectl apply -f k3s/namespace.yaml
3 kubectl apply -f k3s/storageclass.yaml
4 kubectl apply -f k3s/secrets/secret.yaml
5 kubectl apply -f k3s/configmaps/configmap.yaml
6 kubectl apply -f k3s/deployments/
7 kubectl apply -f k3s/services/
8 kubectl apply -f k3s/ingress.yaml
9 kubectl apply -f k3s/backend-hpa.yaml
10 kubectl apply -f k3s/networkpolicy.yaml
11
12 # Verification
13 curl http://<node-ip>:30080 # Acces frontend via NodePort

```

4.3 Vérification des déploiements

4.3.1 3 Deployments distincts (ou StatefulSet pour la DB)

```

kadhém@debian:~$ kubectl get deployments,statefulsets -n tasks-app
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/backend             2/2      2              2            31h
deployment.apps/frontend            2/2      1              2            31h

NAME                                READY    AGE
statefulset.apps/database           1/1      31h

```

FIGURE 4.1 – Replicas

```

1 # V rification des d ploiements
2 kubectl get deployments -n tasks-app
3 kubectl get statefulsets -n tasks-app
4 kubectl get pods -n tasks-app -o wide

```

4.3.2 Resource requests et limits définis sur tous les conteneurs

```

1 # V rification des ressources
2 kubectl describe pod <pod-name> -n tasks-app | grep -A 10 "Limits\|
  Requests"
3 kubectl top pods -n tasks-app

```

4.3.3 Liveness Probe et Readiness Probe configurées

```

1 # V rification des probes
2 kubectl describe pod <pod-name> -n tasks-app | grep -A 5 "Readiness\|
  Liveness"
3 kubectl get events -n tasks-app --field-selector involvedObject.name=<
  pod-name>

```

```

63         resources:
64             requests:
65                 cpu: 100m
66                 memory: 128Mi
67             limits:
68                 cpu: 500m
69                 memory: 256Mi

```

FIGURE 4.2 – Configuration des ressources

```

70         readinessProbe:
71             httpGet:
72                 path: /health
73                 port: 3000
74             initialDelaySeconds: 10
75             periodSeconds: 10
76             timeoutSeconds: 3
77             failureThreshold: 3
78         livenessProbe:
79             httpGet:
80                 path: /health
81                 port: 3000
82             initialDelaySeconds: 20
83             periodSeconds: 20
84             timeoutSeconds: 5
85             failureThreshold: 3

```

FIGURE 4.3 – Liveness et Readiness Probes

4.3.4 Stratégie de déploiement RollingUpdate bien paramétrée

```

1 # Vérification de la stratégie de déploiement
2 kubectl describe deployment backend -n tasks-app | grep -A 5 "Strategy"
3 kubectl rollout status deployment/backend -n tasks-app
4 kubectl rollout history deployment/backend -n tasks-app

```

4.3.5 Conteneurs exécutés en non-root (securityContext)

```

1 # Vérification de l'utilisateur non-root
2 kubectl exec -it <pod-name> -n tasks-app -- whoami
3 kubectl exec -it <pod-name> -n tasks-app -- id
4 kubectl get pod <pod-name> -n tasks-app -o jsonpath='{.spec.
    securityContext}'

```

```

15 spec:
16   replicas: 2
17   strategy:
18     type: RollingUpdate
19     rollingUpdate:
20       maxSurge: 1
21       maxUnavailable: 0

```

FIGURE 4.4 – Configuration RollingUpdate

```

31 spec:
32   securityContext:
33     runAsNonRoot: true
34     runAsUser: 1001
35     runAsGroup: 1001
36     fsGroup: 1001
37     restartPolicy: Always

```

FIGURE 4.5 – SecurityContext - Utilisateur non-root

4.4 Vérification des Services et Exposition

4.4.1 Services ClusterIP pour backend et base de données

```

• kadh@debian:~/devops-m1/M-Haithem/docker-3-tier-tasks$ kubectl get services -n tasks-app
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
backend       ClusterIP   10.43.176.49 <none>        3000/TCP         31h
database      ClusterIP   10.43.12.147 <none>        27017/TCP        31h
database-headless ClusterIP   None         <none>        27017/TCP        31h
frontend      NodePort    10.43.243.85 <none>        80:30080/TCP     31h

```

FIGURE 4.6 – Services ClusterIP

```

1 # Vérification des services ClusterIP
2 kubectl get services -n tasks-app

```

4.4.2 Service pour exposer le frontend (NodePort / LoadBalancer ou Ingress)

```

1 # Vérification du service d'exposition
2 kubectl get service frontend -n tasks-app -o wide

```

```

• kadhem@debian:~/devops-m1/M-Haithem/docker-3-tier-tasks$ kubectl get services -n tasks-app
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
backend       ClusterIP     10.43.176.49   <none>         3000/TCP       31h
database      ClusterIP     10.43.12.147   <none>         27017/TCP      31h
database-headless ClusterIP     None           <none>         27017/TCP      31h
frontend      NodePort      10.43.243.85   <none>         80:30080/TCP   31h
• kadhem@debian:~/devops-m1/M-Haithem/docker-3-tier-tasks$ kubectl get service frontend -n tasks-app
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
frontend      NodePort      10.43.243.85   <none>         80:30080/TCP   31h
• kadhem@debian:~/devops-m1/M-Haithem/docker-3-tier-tasks$ kubectl get ingress -n tasks-app
NAME          CLASS    HOSTS          ADDRESS    PORTS    AGE
tasks-ingress traefik   tasks.local    80         31h
• kadhem@debian:~/devops-m1/M-Haithem/docker-3-tier-tasks$ kubectl get configmaps -n tasks-app
NAME          DATA    AGE
app-config    5        31h
frontend-config 1        31h
kube-root-ca.crt 1        31h
mongo-init-script 1        31h

```

FIGURE 4.7 – Services, engress et configmap

```

3
4 kubectl get ingress -n tasks-app
5
6 # V rification des ConfigMaps
7 kubectl get configmaps -n tasks-app

```

4.4.3 Secrets pour tous les éléments sensibles

```

• kadhem@debian:~/devops-m1/M-Haithem/docker-3-tier-tasks$ kubectl describe secret db-secrets -n tasks-app
Name:         db-secrets
Namespace:    tasks-app
Labels:       <none>
Annotations:  <none>

Type: Opaque

Data
====
DB_APP_PASSWORD:  9 bytes
DB_APP_USER:      7 bytes
DB_ROOT_PASSWORD: 9 bytes
DB_ROOT_USER:     5 bytes

```

FIGURE 4.8 – Secret Base de données - Identifiants sécurisés

```

1 # V rification des Secrets
2 kubectl describe secret db-secrets -n tasks-app

```

Chapitre 5

Tests et Validation

[Section à compléter par l'étudiant]

Cette section doit présenter les tests fonctionnels et de validation effectués sur les deux environnements (Docker Compose et K3s). Elle doit inclure :

- Tests fonctionnels sur Docker Compose (endpoints CRUD, healthchecks, persistance)
- Tests sur le cluster K3s (curl, navigateur, NodePort 30080, Ingress tasks.local)
- Vérification de la haute disponibilité (arrêt d'un worker, résilience des pods)
- Monitoring basique avec `kubect1 get all, logs, describe`
- Captures d'écran et preuves de fonctionnement

Chapitre 6

Difficultés rencontrées et solutions apportées

[Section à compléter par l'étudiant]

Cette section doit documenter les problèmes techniques rencontrés et les solutions mises en œuvre. Exemples de sujets à traiter :

- Problèmes de réseau lors du clonage des VMs (hostname, IP, machine-id)
- Mauvaise sélection d'interface par K3s (node-ip, flannel-iface)
- Rejoindre le cluster après changement de sous-réseau (token, réinstallation agents)
- Problèmes de healthchecks et timeouts
- Persistance des données et volumes

Chapitre 7

Conclusion et perspectives

[Section à compléter par l'étudiant]

Cette section doit inclure :

- Ce qui a été appris durant le projet
- Les compétences acquises en Docker, Kubernetes et DevOps
- Améliorations possibles : CI/CD (GitHub Actions, GitLab CI), monitoring avancé (Prometheus/Grafana), GitOps (ArgoCD), cert-manager pour TLS, multi-réplica MongoDB (replica set)

Annexes

Arborescence complète du projet

[À compléter]

Captures d'écran importantes

[À insérer]

Commandes utiles

[Résumé des commandes kubectl et docker les plus utilisées]